ANAYA

VOICE OF BHARAT

A VOICE-FIRST AI HEALTH ASSISTANT

10 DAYS OF VOICE AGENTS
VOICEFORBHARAT EDITION

DAY 10 – SHARE YOUR VOICE AGENT JOURNEY



BUILT BY
KUNAL CHOUDHARY

HEALTH ACCESS TRACK
Making health conversations more accessible for everyone.

TECHNOLOGY STACK

LiveKit | Deepgram | Google Gemini | Murf Falcon | Python | SQLite

PROJECT LINKS

GitHub Repository
github.com/kunalchwdry/anaya-health-assistant

LinkedIn Profile
linkedin.com/in/kunal-chwdry-8876b0427

LISTENING. UNDERSTANDING. CARING.

ANAYA — VOICE OF BHARAT

Building a Voice-First Health Assistant for Bharat

10 Days of Voice Agents — VoiceForBharat Edition

Built by: Kunal Choudhary
Track: Health Access
Technology: LiveKit • Deepgram • Gemini • Murf Falcon • SIP

Project Overview

A real-time AI voice assistant designed to make health information more accessible through natural voice conversations, multilingual interaction, persistent memory, useful tools, human escalation, outbound calling, and specialist handoffs.

Challenge: 10 Days of Voice Agents
Edition: VoiceForBharat
Day: 10 — Share Your Voice Agent Journey

"Making healthcare conversations more accessible, one voice interaction at a time."

The Problem

Accessing reliable health information can be difficult for people who may not be comfortable with traditional text-based applications. Typing, navigating complex interfaces, or communicating in a non-preferred language can create additional barriers.

Voice offers a more natural way to interact with technology. Instead of searching through menus or typing questions, users can simply speak and receive a spoken response.

During the **10 Days of Voice Agents — VoiceForBharat Edition**, I wanted to explore how conversational AI and voice technology could be combined to create a more accessible health-assistance experience.

This led to **Anaya — Voice of Bharat**, a real-time AI voice assistant designed to help users have natural conversations and access general health information while following appropriate safety guardrails.

Who Is Anaya For?

- People who prefer voice-based interaction
- Users who find typing inconvenient
- Users who prefer Indian languages or code-mixed conversations
- People looking for general health information and guidance
- Users who may need escalation to human assistance

Why Voice?

Natural — Users can communicate by speaking naturally instead of typing.

Accessible — Voice can reduce the barrier created by complex text-based interfaces.

Hands-free — Users can interact with the assistant without constantly typing.

Conversational — The experience feels more like a natural conversation than a traditional chatbot.

Project Goal

The goal of Anaya is not to replace doctors or provide medical diagnoses. Instead, it is designed to provide helpful general health information, guide users toward appropriate next steps, and connect them with human assistance when necessary.

Meet Anaya — Voice of Bharat

Anaya is a real-time AI voice assistant built to make health-related information more accessible through natural voice conversations.

Instead of interacting with a traditional text chatbot, users can simply **speak with Anaya**, ask questions, and receive spoken responses in a conversational experience.

What Anaya Does

Anaya combines conversational AI with real-time voice technology to:

- Understand spoken user queries
- Respond naturally using voice
- Support multilingual and code-mixed conversations
- Remember relevant information for returning users, with permission
- Use tools to provide useful information
- Make outbound calls when required
- Escalate conversations to human assistance
- Hand conversations to specialist agents
- Track conversations through a call analytics dashboard

Built for Bharat

The idea behind Anaya is simple: **make AI-powered assistance feel more natural and accessible for Indian users.**

Voice removes the need to depend entirely on typing and creates an interaction that feels closer to a real conversation.

Anaya uses **Murf Falcon** for fast, natural text-to-speech, helping create a responsive voice experience.

Safety First

Because Anaya operates in the health domain, it follows clear safety boundaries.

It is designed to provide **general health information rather than medical diagnosis or treatment**, and it can involve a human when a situation requires additional assistance.

The Vision

Anaya aims to make accessing helpful health information as simple as having a conversation.

Next: How Anaya's voice pipeline works.

How Anaya Works

Anaya is built as a real-time voice AI pipeline. Several technologies work together to turn a user's speech into an intelligent spoken response.

System Architecture

User → LiveKit → Deepgram → Gemini → Tools & Memory → Murf Falcon → User

1. 🎙️ Speech Input

The user speaks to Anaya through the web interface or phone connection.

2. ⚡ Real-Time Transport — LiveKit

LiveKit manages the real-time audio connection between the user and the voice agent. It allows audio to move between the frontend, backend, and agent with low latency.

3. 📝 Speech-to-Text — Deepgram

Deepgram converts the user's spoken words into text so that the AI model can understand the conversation.

4. 🧠 Intelligence — Gemini

The **Gemini LLM** processes the user's message, conversation context, instructions, and available tools to determine an appropriate response.

5. 🛠️ Tools & Memory

Anaya can use tools when additional information or actions are required. Its memory system can also store permitted information for returning users, making future conversations more useful.

6. 🔊 Text-to-Speech — Murf Falcon

The generated response is sent to **Murf Falcon**, which converts the text into natural speech and sends the audio back to the user.

Complete Flow

```
User speaks
     ↓
LiveKit
     ↓
Deepgram STT
     ↓
Gemini LLM
     ↓
Tools + Memory
     ↓
Murf Falcon TTS
     ↓
User hears response
```

This pipeline allows Anaya to move from **speech → understanding → reasoning → action → speech** in real time.

Technology Stack

Component	Technology
Real-time transport	LiveKit
Speech-to-Text	Deepgram
LLM	Google Gemini
Text-to-Speech	Murf Falcon
Backend	Python
Memory	SQLite
Phone connectivity	SIP
Frontend	LiveKit-based web interface

Key Features of Anaya

Anaya is more than a basic voice chatbot. During the challenge, I focused on building features that make the agent more useful in real-world conversations.

🗣️ 1. Voice-First Conversations

Users can communicate with Anaya by speaking naturally instead of typing. The real-time voice pipeline allows Anaya to listen, process the request, and respond with speech.

🎙️ 2. Voice for Bharat

Anaya is designed with Indian users in mind, including support for multilingual and code-mixed conversations. **Murf Falcon** provides fast, natural-sounding voice output for the assistant.

🧠 3. Persistent Memory

Anaya can remember relevant information for returning users. Memory is handled carefully, and the agent asks for permission before saving personal information.

This allows future conversations to become more personalized without blindly storing everything a user says.

🛠️ 4. AI Tools

Anaya can use tools when a conversation requires information or an action beyond the basic language model response.

This makes the agent more useful than a simple question-and-answer chatbot.

📞 5. Outbound Calling

Anaya can also work beyond the browser through phone-based communication. SIP integration allows the agent to participate in outbound voice calls.

👤 6. Human Escalation

AI should not try to handle every situation alone. Anaya can escalate a conversation when human assistance is more appropriate.

This creates a safer and more practical system.

📊 7. Call Analytics

The project also includes call tracking and analytics to understand conversations and monitor how the voice agent performs.

🔁 8. Specialist Handoffs

When a conversation requires a different type of assistance, Anaya can hand the conversation over to a specialist agent instead of continuing with the wrong context.

What Makes These Features Important?

Together, these features transform Anaya from a simple voice interface into a **complete conversational AI system** capable of remembering users, using tools, making calls, involving humans, and transferring conversations when necessary.

Challenges I Faced

Building a voice agent was not as simple as connecting an LLM to a text-to-speech API. The biggest learning came from debugging the complete real-time system and making different components work together reliably.

Challenge 1 — Connecting the Real-Time Voice Pipeline

One of the main challenges was getting the frontend, LiveKit agent, speech-to-text, LLM, and text-to-speech components to communicate correctly.

At different points, the frontend could connect while the agent was not responding as expected. I had to check the LiveKit connection, environment variables, backend process, and API configuration step by step.

How I Solved It

I tested each part of the pipeline separately and then verified the complete flow:

Frontend → LiveKit → Agent → STT → Gemini → Murf Falcon → User

This helped me identify configuration and connection issues instead of assuming that the problem was with the AI model itself.

What I Learned

A voice agent is a complete system, not just an LLM. A small configuration issue in one component can affect the entire conversation.

Challenge 2 — Persistent Memory

Adding memory introduced another important challenge: the agent should remember useful information without saving everything a user says.

How I Solved It

I implemented persistent storage using **SQLite** and designed the agent to ask for permission before saving user information.

This gave the memory system a clear purpose while keeping user control in the conversation.

What I Learned

Memory should be intentional. A good AI agent should understand **what information is useful to remember and when it should be saved**.

Challenge 3 — Building Beyond the Basic Chat

Adding features such as outbound calls, human escalation, analytics, and specialist handoffs required much more than changing the agent's prompt.

Each feature introduced additional components and new failure points.

How I Solved It

I built and tested the features incrementally instead of trying to implement everything at once. This made it easier to isolate problems and understand how each component interacted with the rest of the system.

What I Learned

The hardest part of building an AI product is often not the AI model itself. It is connecting all the surrounding systems into one reliable experience.

Biggest Takeaway

Build small, test continuously, and debug the entire pipeline—not just the model.

How to Build and Run Anaya

Anaya is built using Python, LiveKit, Deepgram, Google Gemini, Murf Falcon, and SQLite. The project can be used as a starting point for experimenting with real-time voice agents.

Prerequisites

Before running the project, make sure you have:

- Python installed
- Node.js and pnpm installed
- A LiveKit account or local LiveKit setup
- API access for Deepgram
- API access for Google Gemini
- API access for Murf
- Git installed

1. Clone the Repository

Start by cloning the public Anaya repository:

```
git clone https://github.com/kunalchwdry/anaya-health-assistant.git
cd anaya-health-assistant
```

2. Configure Environment Variables

Create the required environment file for the project and add your API credentials.

For example:

```
LIVEKIT_URL=your_livekit_url
LIVEKIT_API_KEY=your_livekit_api_key
LIVEKIT_API_SECRET=your_livekit_api_secret
DEEPGRAM_API_KEY=your_deepgram_api_key
GOOGLE_API_KEY=your_google_api_key
MURF_API_KEY=your_murf_api_key
```

Never publish real API keys in your repository.

Use placeholder values in documentation and keep your actual `.env` files out of Git.

3. Install Dependencies

Install the project's required Python and frontend dependencies using the instructions provided in the repository.

For the frontend, install the required packages with:

```
pnpm install
```

4. Start the Backend

Run the LiveKit agent using the project's Python environment.

```
uv run python src/agent.py
```

The exact command may vary depending on the final project structure.

5. Start the Frontend

In a separate terminal, start the frontend:

```
pnpm dev
```

Open the local development URL shown in the terminal.

6. Connect and Test

Once the frontend and backend are running:

1. Open the Anaya web interface.
2. Connect to the LiveKit session.
3. Allow microphone access.
4. Start speaking with Anaya.
5. Test different conversations and verify that the agent responds correctly.

Important Security Note

Never include API keys, private phone numbers, caller information, personal user data, or private logs in a public repository or blog post.

For a public project, use environment variables and provide only example configuration values.

The complete source code can be inspected in the project's GitHub repository.

Understanding the Setup

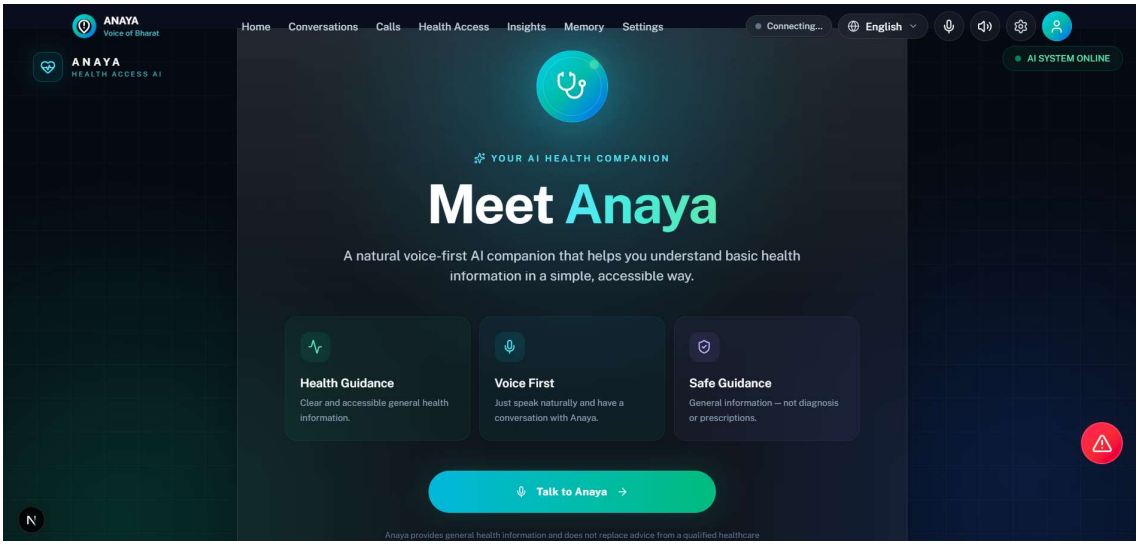
The basic development workflow is:

```
Clone Repository
       ↓
Configure API Keys
       ↓
Install Dependencies
       ↓
Start Agent Backend
       ↓
Start Frontend
       ↓
Connect to LiveKit
       ↓
Talk to Anaya
```

Evidence, Screenshots & Demo

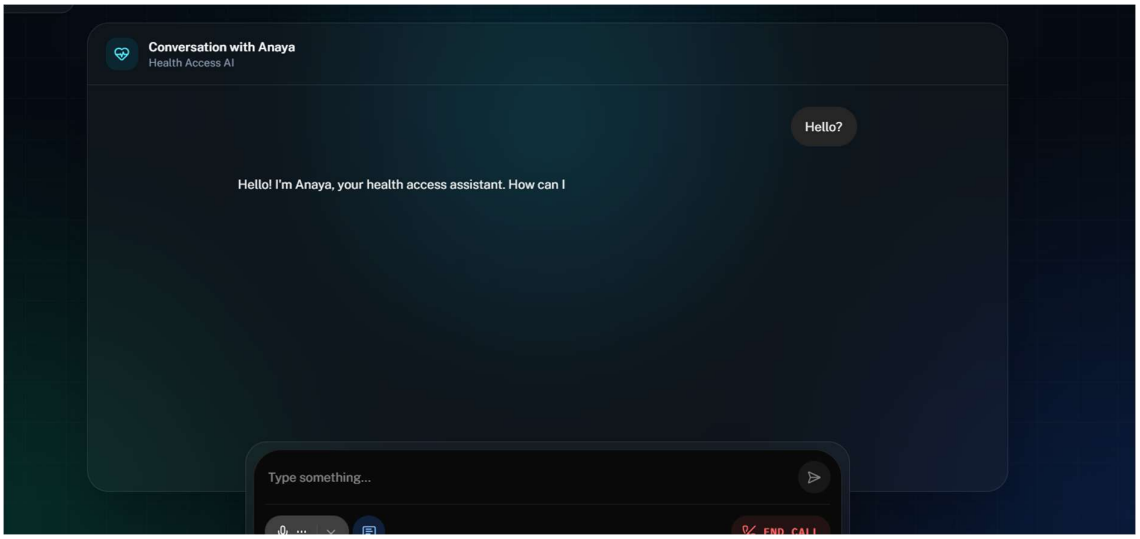
Building Anaya was a hands-on process, so screenshots and demonstrations help show how the system works beyond the written explanation.

Anaya User Interface



This is the main interface through which users connect to Anaya and start a real-time voice conversation.

Voice Conversation



This demonstrates Anaya listening to the user and responding through the real-time voice pipeline.

🔗 **LinkedIn Demo:** https://lnkd.in/p/dXm6RZxy

Outbound Calling

SIP integration allows Anaya to participate in phone-based conversations beyond the browser.

🔗 **LinkedIn Demo:** https://lnkd.in/p/dp4gbtmn

Human Escalation

🔗 **LinkedIn Demo:** https://lnkd.in/p/dy65FQW6

When an AI-only interaction is not appropriate, Anaya can escalate the conversation to human assistance.

Call Analytics

🔗 **LinkedIn Demo:** **https://lnkd.in/p/eT5XJnqY**

The analytics dashboard provides visibility into conversations and call outcomes, helping understand how the agent performs.

Specialist Handoff

Anaya can transfer a conversation to a specialist agent when another agent is better suited to handle the user's request.

🔗 **LinkedIn Demo:** https://lnkd.in/p/dXm6RZxy

Project Repository

The complete source code is publicly available on GitHub:

GitHub: https://github.com/kunalchwdry/anaya-health-assistant

What I Learned

Building Anaya during the 10-day challenge taught me that creating a voice agent is much more than connecting an LLM to a text-to-speech model.

The biggest lesson was understanding how multiple systems have to work together in real time. Speech recognition, the language model, memory, tools, text-to-speech, real-time communication, and the frontend all need to work together as one system.

I also learned that **latency matters greatly in voice applications**. A response that feels acceptable in a text chatbot can feel slow in a voice conversation.

Another important lesson was the importance of **safety and clear boundaries**. Since Anaya operates in the health domain, it should not replace professional medical care. Safety guardrails, human escalation, and specialist handoffs are important parts of the system.

Most importantly, debugging the project taught me to build incrementally. Testing individual components made it easier to identify problems in the complete pipeline.

What I Would Improve Next

Although Anaya is functional, there are several areas I would like to improve:

- Improve response latency for more natural conversations
- Expand multilingual and code-mixed language support
- Add more useful and reliable health-related tools
- Improve call analytics and monitoring
- Make specialist handoffs more intelligent

- Improve the human escalation workflow
- Add more extensive real-world testing
- Deploy the system for wider testing

Final Thoughts

The **10 Days of Voice Agents — VoiceForBharat Edition** challenge gave me the opportunity to move from building a simple conversational agent to thinking about what a complete voice AI product requires.

Anaya can listen, understand, respond, remember permitted information, use tools, make calls, involve humans, and hand conversations to specialists.

My biggest takeaway is:

A useful voice agent is not just an AI that can talk. It is a system designed to listen, understand, act responsibly, and know when it needs help.

Project Links

GitHub Repository:
https://github.com/kunalchwdry/anaya-health-assistant

LinkedIn:
https://www.linkedin.com/in/kunal-chwdry-8876b0427

I have shared the project journey and updates on my LinkedIn. If you'd like to learn more about Anaya or discuss the implementation, feel free to connect with me on LinkedIn.


ANAYA
VOICE OF BHARAT
Thanks for watching.
Built by Kunal Choudhary
LiveKit
Deepgram
Gemini
Murf Falcon